

Redes Neuronales Profundas

Módulo 4 · Deep Learning

Capas densas, activaciones, BatchNorm, conexiones residuales, dropout y el ciclo de entrenamiento en PyTorch con diagnóstico práctico.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Redes Neuronales Profundas

En 2012 AlexNet ganó el concurso ImageNet con un error del 15.3% frente al 26.2% del segundo clasificador. Era la primera vez que una red neuronal profunda superaba con semejante margen a todos los métodos de visión artificial diseñados manualmente. El resultado fue tan decisivo que prácticamente de un año para otro la industria entera migró al deep learning. Trece años después, las redes neuronales profundas son la base de todos los sistemas de IA que importan: los modelos de lenguaje, los sistemas de visión, los motores de recomendación, y los asistentes que procesan millones de consultas al día.

Este tema cubre los fundamentos de las redes neuronales profundas desde sus componentes elementales hasta las técnicas que hacen posible entrenar redes de cientos de capas. No es un tema de programación: es un tema de comprensión de por qué estas arquitecturas funcionan, qué los hace poderosos, y qué condiciones deben darse para que el entrenamiento converja correctamente.

Los componentes fundamentales de una red profunda

Capas densas (fully connected): la unidad básica

Una capa densa conecta cada neurona de entrada con cada neurona de salida a través de pesos aprendibles. Si la capa tiene n entradas y m salidas, tiene $n \times m$ pesos más m sesgos (biases). La operación es: $\text{salida} = \text{activación}(W \times \text{entrada} + b)$. La función de activación introduce la no-linealidad que permite a la red aprender funciones complejas. Sin no-linealidad, apilar capas densas sería equivalente a una única capa lineal.

Funciones de activación: la clave de la no-linealidad

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. Es la activación estándar en capas ocultas de redes profundas. Su derivada es 1 para $x > 0$ y 0 para $x < 0$, lo que evita el problema del gradiente desvaneciente en comparación con sigmoid y tanh. La variante Leaky ReLU usa una pequeña pendiente negativa para $x < 0$ ($f(x) = 0.01x$ si $x < 0$) para evitar las "neuronas muertas" que no aprenden cuando reciben solo valores negativos.

GELU (Gaussian Error Linear Unit) y SiLU son las activaciones modernas en Transformers y redes de nueva generación. A diferencia de ReLU, son suaves (diferenciables en todo punto), lo que puede mejorar la estabilidad del entrenamiento. GPT-2 usa GELU; Llama usa SiLU.

Normalización de capas: estabilizar el entrenamiento

Batch Normalization (BatchNorm) normaliza las activaciones de cada capa por el batch actual, forzando que tengan media 0 y varianza 1. Esto permite usar tasas de aprendizaje más altas y hace el entrenamiento más estable. Layer Normalization (LayerNorm) normaliza por las características de cada ejemplo individual en lugar de por el batch: es la variante usada en Transformers porque funciona mejor con secuencias de longitud variable. RMSNorm es la variante más eficiente, usada en Llama: normaliza solo por la raíz de la media de los cuadrados,

sin centrar la media.

Conexiones residuales: el breakthrough que hizo posibles las redes muy profundas

Las conexiones residuales (skip connections o residual connections), introducidas en ResNet (He et al., 2016), permiten al gradiente fluir directamente desde las capas más profundas a las más superficiales sin pasar por las transformaciones intermedias. La fórmula es simple: $\text{output} = \text{capa}(\text{input}) + \text{input}$. Este "atajo" hace posible entrenar redes de 100, 1000 o más capas sin el problema del gradiente desvaneciente. Son también la razón por la que los Transformers pueden ser muy profundos: cada bloque del Transformer usa una conexión residual que preserva la información de la capa anterior.

SECCIÓN 3 · CÓMO FUNCIONA

El ciclo de entrenamiento: forward pass, loss, backward pass, update

El entrenamiento de una red neuronal profunda sigue un ciclo iterativo. Forward pass: los datos de entrada fluyen hacia adelante a través de todas las capas, cada una aplicando su transformación ($W \times x + b \rightarrow \text{activación}$), produciendo una predicción en la capa de salida. Loss computation: se calcula la función de pérdida (cross-entropy para clasificación, MSE para regresión) comparando la predicción con el target real. Backward pass (backpropagation): el gradiente de la pérdida se propaga hacia atrás por todas las capas usando la regla de la cadena. Parameter update: el optimizador (Adam, SGD) actualiza los pesos en la dirección que reduce la pérdida.

En PyTorch, este ciclo es explícito y controlable: `outputs = model(inputs)`, `loss = criterion(outputs, labels)`, `optimizer.zero_grad()`, `loss.backward()`, `optimizer.step()`. Esta explicitud es una de las razones por las que PyTorch se ha convertido en el framework estándar tanto en investigación como en producción: da control total sobre cada paso del entrenamiento.

Dropout: regularización por desactivación aleatoria

Durante el entrenamiento, dropout desactiva aleatoriamente una fracción p de las neuronas en cada forward pass. Esto fuerza a la red a no depender de ninguna neurona específica, aprendiendo representaciones más robustas y redundantes. Durante la inferencia, todas las neuronas están activas pero sus salidas se escalan por $(1-p)$ para compensar. Las tasas más comunes son $p=0.1$ a $p=0.5$.

SECCIÓN 4 · EJEMPLOS REALES

- AlexNet (2012): 5 capas convolucionales + 3 densas, 60M parámetros, error ImageNet 15.3%. Primer uso masivo de GPUs para deep learning. Marcó el inicio de la era del deep learning.
- ResNet-50 (2016): 50 capas con conexiones residuales, 25M parámetros, error ImageNet ~3.8%. La arquitectura residual permitió redes mucho más profundas que antes eran

imposibles de entrenar. Base de infinitos sistemas de visión en producción.

- Redes neuronales en detección de fraude en Stripe: redes densas profundas con capas de 512-2048 neuronas, activaciones ReLU, BatchNorm y dropout. Procesan decenas de features de cada transacción para producir un score de fraude en <10ms.
- EfficientNet en clasificación de imágenes médicas: escala de forma equilibrada profundidad, anchura y resolución. Usado en sistemas de detección de retinopatía diabética con precisión comparable a oftalmólogos especializados.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Más capas no siempre es mejor. Sin conexiones residuales, las redes muy profundas sufren el problema del gradiente desvaneciente y pueden tener peor rendimiento que redes más superficiales. La profundidad solo añade capacidad si va acompañada de BatchNorm o LayerNorm y conexiones residuales.

Error 2: Olvidar el dropout en entrenamiento y activarlo en inferencia es un error crítico. PyTorch requiere llamar explícitamente a `model.eval()` antes de la inferencia, lo que desactiva el dropout y cambia el comportamiento de BatchNorm. Inferir con `model.train()` activo produce predicciones estocásticas incorrectas.

Error 3: Usar redes neuronales densas para datos tabulares sin comparar con gradient boosting. Las redes densas generalmente no superan a XGBoost en datos tabulares medianos. Solo cuando hay millones de ejemplos o relaciones muy complejas entre variables puede una red neuronal superar a XGBoost en datos tabulares.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para construir una red neuronal densa básica en PyTorch: define la arquitectura con `nn.Sequential` o subclasificando `nn.Module`, inicializa los pesos (PyTorch usa inicialización de Kaiming por defecto para ReLU, que es correcta), define la función de pérdida y el optimizador, implementa el ciclo de entrenamiento con `forward/backward/update`, y evalúa regularmente en el conjunto de validación con `model.eval()`. El framework estándar para este proceso es PyTorch Lightning, que encapsula el boilerplate del ciclo de entrenamiento y proporciona callbacks, logging y checkpointing automáticos.

Para diagnóstico de entrenamiento: monitoriza la pérdida de entrenamiento y validación en tiempo real con Weights & Biases (wandb) o TensorBoard. Si la pérdida de validación no baja aunque la de entrenamiento sí: aumenta dropout o regularización. Si la pérdida de entrenamiento tampoco baja: aumenta la capacidad de la red, revisa la tasa de aprendizaje, o revisa la calidad de los datos.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M2-T2 (Backpropagation): el ciclo de entrenamiento descrito aquí es la aplicación práctica de la teoría de backpropagation del Módulo 2.
- M4-T2 (CNNs): las CNNs son redes profundas especializadas para datos con estructura espacial, construidas sobre los mismos principios que se introducen aquí.

- M4-T5 (Transfer learning): el transfer learning toma redes profundas preentrenadas y las adapta a nuevas tareas.

SECCIÓN 8 · RESUMEN EJECUTIVO

Las redes neuronales profundas son secuencias de capas con pesos aprendibles separadas por funciones de activación no lineales. Los componentes clave son: capas densas o convolucionales, activaciones (ReLU, GELU, SiLU), normalización (BatchNorm, LayerNorm), conexiones residuales (skip connections) y dropout para regularización. El entrenamiento sigue el ciclo forward pass → loss → backprop → update. PyTorch es el framework estándar (model.train() para entrenar, model.eval() para inferir). Herramientas de diagnóstico: curvas de pérdida en wandb o TensorBoard. Escala progresivamente la capacidad de la red monitorizando la brecha train/val.